# Git — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

## Contents


### Beginner (6 topics)

01 Introduction to Version Control

02 Installation and Configuration

03 Creating a Repository

04 The Three Areas: Working Tree, Staging, Repository

05 Status, Log and Diff

06 Ignoring Files and Undoing Changes

### Intermediate (6 topics)

07 Branching and Merging

08 Resolving Merge Conflicts

09 Remotes: fetch, pull, push

10 Tags and Releases

11 Stash

12 Collaboration Workflows

### Advanced (6 topics)

13 Rebase and Interactive Rebase

14 Cherry-pick and Revert

15 Reset Modes and Reflog

16 Bisect and Blame

17 Submodules and Worktrees

18 Git Hooks

## Master (4 topics)


| | |
|---|---|
| 19 | Git Internals |
| 20 | Large Repos, Monorepos and LFS |
| 21 | Security: Signed Commits and Secrets |
| 22 | Best Practices |

# Beginner Level

## 01. Introduction to Version Control

**DEFINITION**

Git tracks changes to files so you can go back in time, work in parallel and collaborate. It is distributed: every clone has the full history.

**SYNTAX**

```
git --version
```

**EXAMPLE**

```
git --version
```

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

A school notebook where every correction is dated and kept, so you can see old versions any time; Git saves the history of your files.

**தமிழ்**

ஒவ்வொரு திருத்தமும் தேதியுடன் சேமிக்கப்படும் பள்ளி நோட்டுப் புத்தகம் போல Git கோப்புகளின் வரலாற்றைச் சேமிக்கிறது.

**PROS**

- Complete history
- Works offline
- Fast and distributed
- Branching is cheap

**CONS**

- Learning curve
- Large binary files are awkward
- Rewriting history can be dangerous

**ADVANCED / NEW FEATURES**

- SHA-256 repositories
- Reftable backend (2.45)
- `git maintenance` and Scalar for big repos
- Git 3.0 plans `main` as default branch

## 02. Installation and Configuration

**DEFINITION**

Install Git, then set your identity and default branch.

**SYNTAX**

```
git config --global user.name "Name"
```

**EXAMPLE**

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

Writing your name on your exam paper so everyone knows who wrote it; configuration tells Git who you are.

**தமிழ்**

தேர்வுத் தாளில் உங்கள் பெயரை எழுதுவது போல configuration நீங்கள் யார் என்பதை Git-க்குச் சொல்கிறது.

**PROS**

- One-time setup
- Per-repo overrides

**CONS**

- Wrong email links commits to wrong account

**ADVANCED / NEW FEATURES**

- `init.defaultBranch`
- SSH signing support

## 03. Creating a Repository

**DEFINITION**

`git init` starts a repo; `git clone` copies an existing one.

**SYNTAX**

```
git init
git clone <url>
```

**EXAMPLE**

```
git init my-project
```

```
git clone https://github.com/user/repo.git
```

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

Starting a new notebook or photocopying a friend's whole notebook; init creates a repo, clone copies one.

**தமிழ்**

புதிய நோட்டுப் புத்தகம் தொடங்குவது அல்லது நண்பரின் முழு நோட்டையும் நகலெடுப்பது போல init repo உருவாக்கும், clone நகலெடுக்கும்.

**PROS**

- Instant repo creation
- Full history in every clone

**CONS**

- Large clones are slow

**ADVANCED / NEW FEATURES**

- Partial and shallow clones
- `--filter=blob:none`

## 04. The Three Areas: Working Tree, Staging, Repository

**DEFINITION**

Edit files (working tree), choose changes with `git add` (staging), save a snapshot with `git commit`.

**SYNTAX**

```
git add .
git commit -m "msg"
```

**EXAMPLE**

```
git add .
git commit -m "Add login page"
```

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

Choosing which finished pages to put in an envelope (staging) and sealing it with a note (commit).

**தமிழ்**

எந்த முடிந்த பக்கங்களை உறையில் வைக்க வேண்டும் என்று தேர்ந்தெடுத்து (staging) குறிப்புடன் ஒட்டுவது (commit) போல.

**PROS**

- Fine-grained commits
- Review before saving

**CONS**

- Staging area confuses beginners

**ADVANCED / NEW FEATURES**

- `git add -p` for partial staging
- `git restore --staged`

## 05. Status, Log and Diff

**DEFINITION**

Inspect your work and history.

**SYNTAX**

```
git status
git log --oneline
git diff
```

**EXAMPLE**

```
git status
git log --oneline --graph --all
git diff
```

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

Checking the notebook to see what changed since last time; status, log and diff show your work.

**தமிழ்**

கடந்த முறைக்குப் பிறகு என்ன மாறியது என்று நோட்டில் பார்ப்பது போல status, log, diff உங்கள் வேலையைக் காட்டுகின்றன.

**PROS**

- Clear visibility
- Powerful filtering

**CONS**

- Output can be overwhelming

### ADVANCED / NEW FEATURES

- `git log --graph`
- `git diff --word-diff`

## 06. Ignoring Files and Undoing Changes

### DEFINITION

`.gitignore` hides files; `restore` and `reset` undo work safely.

### SYNTAX

```
git restore f
git reset --soft HEAD~1
```

### EXAMPLE

```
git restore file.txt
git restore --staged file.txt
git commit --amend
```

### REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

**English**

Using an eraser on the page you just wrote; restore and reset undo mistakes safely.

**தமிழ்**

இப்போது எழுதிய பக்கத்தை அழிப்பான் கொண்டு அழிப்பது போல restore, reset தவறுகளை பாதுகாப்பாக மீட்கின்றன.

### PROS

- .gitignore keeps junk out
- Many undo options

### CONS

- `reset --hard` can destroy work

### ADVANCED / NEW FEATURES

- `git switch` and `git restore` (2.23) replace overloaded `checkout`

# Intermediate Level

## 07. Branching and Merging

**DEFINITION**

Branches let you work in isolation; merge to combine. Fast-forward vs merge commit.

**SYNTAX**

```
git switch -c feature
git merge feature
```

**EXAMPLE**

```
git switch -c feature/login
git switch main
git merge feature/login
```

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

Writing a draft on a separate page and later copying it into the final notebook; branches isolate work.

**தமிழ்**

தனிப் பக்கத்தில் வரைவு எழுதி பின்னர் இறுதி நோட்டில் சேர்ப்பது போல branches வேலையைத் தனிமைப்படுத்துகின்றன.

**PROS**

- Cheap branches
- Parallel work
- Safe experiments

**CONS**

- Long-lived branches cause conflicts

**ADVANCED / NEW FEATURES**

- `git switch`
- `--update-refs` for stacked branches (2.38)

## 08. Resolving Merge Conflicts

**DEFINITION**

Edit conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`), then add and commit.

### SYNTAX

```
<<<<<<< HEAD
...
=======
...
>>>>>>> branch
```

### EXAMPLE

```
git status   # see conflicted files
git add file && git commit
```

### REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

**English**

Two friends edit the same line of a shared notebook; you decide which version to keep.

**தமிழ்**

பகிரப்பட்ட நோட்டின் ஒரே வரியை இரு நண்பர்கள் திருத்தினால் எதை வைத்துக்கொள்வது என்று நீங்கள் தீர்மானிக்கிறீர்கள்.

### PROS

- Explicit resolution
- Merge tools help

### CONS

- Time consuming on big conflicts

### ADVANCED / NEW FEATURES

- `merge.conflictStyle zdiff3`
- `git rerere` remembers resolutions

## 09. Remotes: fetch, pull, push

### DEFINITION

Sync with a remote repository like GitHub.

### SYNTAX

```
git push -u origin main
git pull --rebase
```

### EXAMPLE

```
git remote add origin <url>
git push -u origin main
git pull --rebase
```

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

Sending your notebook pages to the common cupboard and fetching others' pages; remotes sync work with a team.

**தமிழ்**

உங்கள் நோட்டுப் பக்கங்களை பொது அலமாரிக்கு அனுப்பி மற்றவர்களுடையவற்றை எடுப்பது போல remotes குழுவுடன் வேலையை ஒத்திசைக்கின்றன.

**PROS**

- Collaboration
- Backup

**CONS**

- Force-push can overwrite others' work

**ADVANCED / NEW FEATURES**

- `--force-with-lease`
- Push options
- `git pull --rebase` config

## 10. Tags and Releases

**DEFINITION**

Mark versions with tags.

**SYNTAX**

```
git tag -a v1.0 -m "msg"
```

**EXAMPLE**

```
git tag -a v1.0.0 -m "First release"
git push origin v1.0.0
```

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

Putting a bookmark on the page where the final exam draft was completed; tags mark releases.

**தமிழ்**

இறுதித் தேர்வு வரைவு முடிந்த பக்கத்தில் புக்மார்க் வைப்பது போல tags வெளியீடுகளைக் குறிக்கின்றன.

**PROS**

- Permanent version markers
- Easy releases

**CONS**

- Tags must be pushed explicitly

**ADVANCED / NEW FEATURES**

- Signed tags
- Release automation

## 11. Stash

**DEFINITION**

Temporarily shelve changes.

**SYNTAX**

```
git stash push
git stash pop
```

**EXAMPLE**

```
git stash push -m "wip"
git stash pop
```

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

Keeping your half-written page in a drawer to quickly do an urgent task; stash shelves changes temporarily.

**தமிழ்**

அவசர வேலைக்காக பாதி எழுதிய பக்கத்தை இழுப்பறையில் வைப்பது போல stash மாற்றங்களை தற்காலிகமாக ஒதுக்குகிறது.

**PROS**

- Quick context switch

**CONS**

- Easy to forget stashes

**ADVANCED / NEW FEATURES**

- `git stash --include-untracked`
- `git stash push -m`

## 12. Collaboration Workflows

**DEFINITION**

Feature branches, pull/merge requests, code review; compare GitHub Flow and Git Flow.

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

A group assignment: everyone works on their part and the teacher reviews before the final submission; PR workflows do the same.

**தமிழ்**

குழு பணி: ஒவ்வொருவரும் தங்கள் பகுதியைச் செய்து இறுதிச் சமர்ப்பிப்புக்கு முன் ஆசிரியர் பார்வையிடுவது போல PR workflows செயல்படுகின்றன.

**PROS**

- Code review culture
- Safer main branch

**CONS**

- Slow reviews block progress

**ADVANCED / NEW FEATURES**

- Merge queues
- Trunk-based development

# Advanced Level

## 13. Rebase and Interactive Rebase

**DEFINITION**

Rebase replays commits on a new base for a linear history; interactive rebase squashes/edits commits (only on unpublished work).

**SYNTAX**

```
git rebase main
git rebase -i HEAD~3
```

**EXAMPLE**

```
git rebase main
git rebase -i HEAD~3
```

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

Re-writing your draft on fresh pages in a clean order before showing it; rebase makes history linear.

**தமிழ்**

காட்டும் முன் வரைவை புதிய பக்கங்களில் ஒழுங்கான வரிசையில் மீண்டும் எழுதுவது போல rebase வரலாற்றை நேர்கோடாக்குகிறது.

**PROS**

- Clean linear history
- Squash fixup commits

**CONS**

- Never rebase shared history
- Conflicts replay per commit

**ADVANCED / NEW FEATURES**

- `--autosquash`
- `--update-refs`
- `rebase.updateRefs`

## 14. Cherry-pick and Revert

**DEFINITION**

Copy one commit to another branch; create a commit that undoes another.

#### SYNTAX

```
git cherry-pick <sha>
git revert <sha>
```

#### EXAMPLE

```
git cherry-pick <sha>
git revert <sha>
```

#### REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

**English**

Copying one good page into another notebook, or writing a note that cancels an earlier page.

**தமிழ்**

ஒரு நல்ல பக்கத்தை மற்றொரு நோட்டில் நகலெடுப்பது அல்லது முந்தைய பக்கத்தை ரத்து செய்யும் குறிப்பு எழுதுவது போல.

#### PROS

- Port a fix between branches
- Safe undo with revert

#### CONS

- Duplicate commits across branches

#### ADVANCED / NEW FEATURES

- Multi-commit cherry-pick ranges

## 15. Reset Modes and Reflog

#### DEFINITION

`--soft`, `--mixed`, `--hard`; recover lost commits with `reflog`.

#### SYNTAX

```
git reflog
git reset --hard HEAD@{2}
```

#### EXAMPLE

```
git reflog
git reset --hard HEAD@{2}
```

#### REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

**English**

A CCTV recording of every move you made, so you can recover even deleted work; reflog records HEAD movements.

**தமிழ்**

நீங்கள் செய்த ஒவ்வொரு நகர்வையும் பதிவு செய்யும் CCTV போல reflog நீக்கிய வேலையையும் மீட்க உதவுகிறது.

**PROS**

- Recovery safety net

**CONS**

- Entries expire

**ADVANCED / NEW FEATURES**

- `git reflog expire` tuning

## 16. Bisect and Blame

**DEFINITION**

Find the commit that introduced a bug; see who changed each line.

**SYNTAX**

```
git bisect start
git blame file
```

**EXAMPLE**

```
git bisect start
git blame app.js
```

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

Finding on which day the milk got spoiled by checking days one by one (binary search); bisect finds the commit that broke code.

**தமிழ்**

எந்த நாள் பால் கெட்டது என்று நாட்களை பாதியாகப் பிரித்துக் கண்டுபிடிப்பது போல bisect குறியீட்டை உடைத்த commit-ஐக் கண்டுபிடிக்கிறது.

**PROS**

- Finds bugs fast
- Automatable with scripts

**CONS**

- Needs reliable tests

#### ADVANCED / NEW FEATURES

- `git bisect run` automation

## 17. Submodules and Worktrees

#### DEFINITION

Embed other repos; work on several branches at once.

#### SYNTAX

```
git worktree add ../x branch
```

#### EXAMPLE

```
git worktree add ../hotfix hotfix
```

#### REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

**English**

Opening two notebooks on the table at once instead of closing one; worktrees check out several branches together.

**தமிழ்**

ஒன்றை மூடாமல் மேசையில் இரண்டு நோட்டுகளைத் திறந்து வைப்பது போல worktrees பல branches-ஐ ஒரே நேரத்தில் கொண்டிருக்கும்.

#### PROS

- Parallel branches
- No stashing

#### CONS

- More disk space

#### ADVANCED / NEW FEATURES

- Worktree improvements
- Submodule recursion fixes

## 18. Git Hooks

#### DEFINITION

Run scripts on commit/push (lint, tests); use Husky or pre-commit.

#### SYNTAX

```
.git/hooks/pre-commit
```

### EXAMPLE

```
.git/hooks/pre-commit
```

### REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

**English**

A security check at the gate before anyone enters; hooks run scripts on commit or push.

**தமிழ்**

உள்ளே நுழையும் முன் வாயிலில் பாதுகாப்புச் சோதனை போல hooks commit அல்லது push-இல் scripts இயக்குகின்றன.

### PROS

- Automated checks
- Consistent quality

### CONS

- Hooks are not shared by default (use Husky/pre-commit)

### ADVANCED / NEW FEATURES

- `core.hooksPath`
- pre-commit framework

# Master Level

## 19. Git Internals

**DEFINITION**

Objects: blobs, trees, commits, refs; content-addressed storage with SHA hashes.

**SYNTAX**

```
git cat-file -p HEAD
```

**EXAMPLE**

```
git cat-file -p HEAD
```

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

Opening the safe to see how the lock works: Git stores blobs, trees and commits as hashed objects.

**தமிழ்**

பூட்டு எப்படி வேலை செய்கிறது என்று பார்க்க பெட்டியைத் திறப்பது போல Git blobs, trees, commits-ஐ hash செய்யப்பட்ட objects ஆக சேமிக்கிறது.

**PROS**

- Understanding helps debugging
- Content-addressed integrity

**CONS**

- Low-level and technical

**ADVANCED / NEW FEATURES**

- SHA-256 transition
- Commit-graph and bitmap indexes

## 20. Large Repos, Monorepos and LFS

**DEFINITION**

Shallow and sparse clones, partial clone, Git LFS for big files.

**SYNTAX**

```
git clone --depth 1 <url>
git lfs track "*.psd"
```

**EXAMPLE**

```
git clone --depth 1 <url>
git lfs track "*.psd"
```

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

Carrying only the latest page instead of the whole library, and storing heavy photos in a separate warehouse; shallow clones and LFS handle big repos.

**தமிழ்**

முழு நூலகத்துக்குப் பதிலாக சமீபத்திய பக்கத்தை மட்டும் எடுத்துச் செல்வது, கனமான படங்களை தனிக் கிடங்கில் வைப்பது போல shallow clone, LFS பெரிய repos-ஐக் கையாளுகின்றன.

**PROS**

- Faster clones
- Large file support

**CONS**

- LFS needs server support and quota

**ADVANCED / NEW FEATURES**

- Sparse checkout cone mode
- Partial clone
- `git sparse-checkout`

## 21. Security: Signed Commits and Secrets

**DEFINITION**

Sign commits with GPG/SSH; never commit secrets; clean history with `git filter-repo`.

**SYNTAX**

```
git config commit.gpgsign true
```

**EXAMPLE**

```
git config --global commit.gpgsign true
```

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

Signing your answer sheet so nobody can claim it; signed commits prove who authored them.

**தமிழ்**

உங்கள் விடைத்தாளில் கையொப்பமிடுவது போல signed commits யார் எழுதினார் என்பதை நிரூபிக்கின்றன.

**PROS**

- Authenticity
- Verified badges on hosts

**CONS**

- Key management overhead

**ADVANCED / NEW FEATURES**

- SSH-key signing
- Secret scanning push protection

## 22. Best Practices

**DEFINITION**

Small focused commits, meaningful messages (Conventional Commits), protected main, frequent pulls, never rewrite shared history.

**SYNTAX**

```
feat(auth): add login
```

**EXAMPLE**

```
feat(auth): add password reset
```

**REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)**

**English**

Writing neat headings in your notebook so anyone can understand later; good commit messages and small commits help teams.

**தமிழ்**

பின்னர் யாரும் புரிந்துகொள்ள நோட்டில் அழகான தலைப்புகள் எழுதுவது போல நல்ல commit செய்திகள், சிறிய commits குழுவுக்கு உதவுகின்றன.

**PROS**

- Readable history
- Automatic changelogs

**CONS**

- Needs team discipline

## ADVANCED / NEW FEATURES

- Conventional Commits
- Git 3.0 direction: defaults like `main` branch

## Interview Question Bank

**Q1. Git vs GitHub?**
Git is the version control tool; GitHub is a hosting platform for Git repositories.

**Q2. `git pull` vs `git fetch`?**
`fetch` downloads changes; `pull` = fetch + merge (or rebase).

**Q3. Merge vs rebase?**
Merge preserves history with a merge commit; rebase rewrites commits for a linear history.

**Q4. What is HEAD?**
A pointer to the current commit/branch.

**Q5. `reset` vs `revert`?**
`reset` moves the branch pointer (rewrites history); `revert` adds a new commit that undoes changes.

**Q6. What is a detached HEAD?**
Checking out a commit instead of a branch; new commits are not on any branch.

**Q7. How do you undo the last commit?**
`git reset --soft HEAD~1` keeps changes; `git revert HEAD` is safe for shared branches.

**Q8. What is `git stash`?**
Saves uncommitted changes temporarily so you can switch tasks.

**Q9. What is a fast-forward merge?**
The branch pointer just moves ahead because there are no diverging commits.

**Q10. How do you resolve conflicts?**
Edit the conflicted files, stage them, and complete the merge/rebase.

## Capstone Project Ideas

1. Maintain a personal portfolio repo with branches
2. Practice conflicts with a friend
3. Create a release with tags
4. Set up a pre-commit hook
5. Contribute to an open-source repo